Welcome to "The Advanced Digital World"

50 Min. Class Format

- 5Min.: Pre-class Q&As
- 5 Min.: Review of the Previous Class
- 20 Min.: Lecture part 1
- 5 Min.: Administration
- 15 Min.: Lecture part 2
- 5 Min.: Summary/Q&As
- 5 Min.: Post-class Q/As (or @ my office if necessary)

VHDL

- VHSIC (Very High Speed IC) Hardware Description Language
- IEEE Standard: 1076 & 1164 (1987)
- Similar to 'ADA' / 'Fortran'
- What is different from other program languages?
  - Parallel operations
  - Delay: variable/signal
  - Simulation/Synthesis

Evolution of VHDL

- IEEE-1164 ('87) defines the 9-value logic types: std_ulogic & std_ulogic_vector.
- IEEE-1076 ('87) includes a wide range of data types(e.g., integer/real, bit/boolean, character/time, & bit_vector/string.
  - IEEE-1076('93) adds the 'xnor' operator, etc.
  - IEEE-1076.1 provids analog and mixed-signal circuit design extensions.
  - IEEE-1076.2 adds better handling of real and complex data types.
  - IEEE-1076.3 introduces signed/unsigned types on vectors.

Verilog HDL

- Widely used in industry
- Popular at the low-levels of design
  - From RTL to gate/switch level
- IEEE standard (1995)
- Similar to 'C'
- Easy to learn

5

VHDL vs Verilog

- Case sensitivity
  - VHDL: insensitive
- Case Statements
  - VHDL: mutually exclusive → no priority logic
  - VHDL: cover all the different values in 'Case'
- Signed data types
  - VHDL: signed/unsigned data type
  - Verilog: Integer type: signed / register type: unsigned

6

VHDL vs Verilog (Cont.)

- I/O Ports
  - VHDL: in, out, inout, buffer
  - Verilog: input, output, inout
- Out Port access
  - VHDL: not allowed → need to create a signal
  - Or using 'buffer' → read/write, but not driven
  - Verilog: output port creates a temporary signal

7

VHDL vs Verilog (Cont.)

- Components
  - VHDL: declaration → instantiation
- Multiple Drivers
  - VHDL: user/STD resolution functions

8

VHDL Syntax

- Library
- Entity
- Architecture
- Concurrent Statements
  - Process
  - concurrent signal assignment
  - concurrent procedure call (SW black box)
  - component instantiation statement (HW black box)
- Data Types/subtypes: std_logic_vector/etc.
- Operators: log/arith/shf/rel/etc.
- Control structures: if/case/loop/for/while/etc.

VHDL Syntax (Library)

- Include std & working libs
- Syntax
  - library IEEE;
  - use IEEE.Std_Logic_1164.all; ← std library
  - use std.TextIO.all; ← std library
  - use work.all; ← user working library

VHDL Syntax (Entity)

- define interface
- represent a design
- Syntax
  - entity IDENTIFIER is
    - Header
    - Port
    - type/constant/alias/file/etc.
  - begin
  - …
  - end IDENTIFIER;

VHDL Syntax (Entity) Example

```
entity DECODER is
  generic (Maxcount: natural := 5;
           MaxDelay: time := 50 ns);

  port (    SUM : in STD_LOGIC_VECTOR (3 downto 0);
          INDATA : out STD_LOGIC_VECTOR (15 downto 0);
          ODATA : out STD_LOGIC);

  :
  :
end DECODER;
```

VHDL Syntax (Entity) Example

```
entity DECODER is
   :
   :
   type State_type is (Idle, Ready, Running, Stall);
   constant Upper_Limit_c : integer := 32;

   alias OpCode : STD_LOGIC_VECTOR (7 downto 0) is Data (15
   downto 8);
   alias SRC : STD_LOGIC_VECTOR (4 downto 0) is Data (7
   downto 4);
   alias DST : STD_LOGIC_VECTOR (4 downto 0) is Data (3
   downto 0);
end DECODER;
```


VHDL Syntax (Entity) Example

```
entity DECODER is
   generic (Maxcount: natural := 5;
            MaxDelay: time := 50 ns);

   port (    SUM : in STD_LOGIC_VECTOR (3 downto 0);
           INDATA : out STD_LOGIC_VECTOR (15 downto 0);
           ODATA : out STD_LOGIC);

   type State_type is (Idle, Ready, Running, Stall);
   constant Upper_Limit_c : integer := 32;

   alias OpCode : STD_LOGIC_VECTOR (7 downto 0) is Data (15 downto
   8);
   alias SRC : STD_LOGIC_VECTOR (4 downto 0) is Data (7 downto 4);
   alias DST : STD_LOGIC_VECTOR (4 downto 0) is Data (3 downto 0);
end DECODER;
```


VHDL Syntax (Architecture)

- What for?
  - Internal organization or operation
- What kinds?
  - Behavior: function
  - Data flow: data processing flow
  - Structure: interface/design hierarchy

Architecture (Syntax)

- architecture a_identifier of entity_name is
  - Architecture declarative part
    - Block declarative items
      - Procedures
      - Components
      - Types/subtypes
      - Constants/signals/shared variables
      - Files/aliases/attributes/etc.
- begin
  - Architecture statement part
- end a_identifier;

Concurrent Statements

- Interconnects blocks and processes
  - → describe overall behavior/structure
- Execute asynchronously
  - → order of definitions is irrelevant
- How to synchronize them?
  - → use sensitivity clauses or wait statements

Concurrent Statements

They are:

- Process
  - Process ZZZ (A1, A2, …)
- Concurrent signal assignment
  - S1 <= '1' after 5 ns;
- Concurrent procedure call (SW black box)
  - Con_pkg.setup (src1 => "IN1", clk => CLK, dat1 => DT1);

Signal/variable Assignment

- signal i : integer := 0;
- signal j : integer := 2;
- signal k : integer := 3;
- What is the value of "i"?
  - Concurrent signal assignment
  - i <= j + k;
  - i <= i + j;
  - i =
- variable i : integer := 0;
- variable j : integer := 2;
- variable k : integer := 3;
- What is the value of "i"?
  - Variable assignment
  - i := j + k;
  - i := i + j;
  - i =

Concurrent Statements

They are (cont.):

- component instantiation statement (HW black box)
  - Component XOR_1
  - Port (IN1, IN2 : in std_logic;
  - OUT1: out std_logic);

Concurrent Statements

They are (cont.):

- generate statement (duplicate multiple comp)
  - BK: For J_i in 0 to 7 generate
  - BK0: if J_i = 0 generate
  - BK_XOR: XOR_1
  - Port map (IN1 => BKin (J_i),
  - IN2 => BKin (J_i+1),
  - OUT1 => BKout(J_i));

21

Process

Characteristics

- Each one is independent
- Inside process, it is sequential
- The behavior of the design
- Flexibility: easy to understand sequential nature
- Synchronization: with other CSs using 'wait statement' or 'sensitivity clause'

22

Process (Syntax)

- process_label: (optional) process (sensitivity_list)
  - process_declarative_part
- begin
  - process_statement_part
- end process process_label (optional) ;

23

Process: Example 1

- With sensitivity list
  - Example_1: process (ENTC_S1, ENTC_S2)
  - begin
    - ENTC_S1 <= '0';
    - ENTC_S2 <= '1' after 20 ns;
  - end process Example_1;

24

Process: Example 2

- With a wait statement
  - Example_1: process
  - begin
    - ENTC_S1 <= '0';
    - ENTC_S2 <= '1' after 20 ns;
    - Wait on ENTC_S1, ENTC_S2;
  - end process Example_1;

Process: Example 3

- Caution: No 'wait' with 'sensitivity' clause
  - Example_2: process (ENTC_S1, ENTC_S2)
  - begin
    - ENTC_S1 <= '0';
    - ENTC_S2 <= '1' after 20 ns;
    - Wait for 50 ns; ← *'coding error'*
  - end process Example_2;

Process: Example 3 (cont.)

- Only static signal names for sensitivity list
  - Signal S1: integer := 4;
  - ENTC_S1 → static signal name
  - ENTC_S16 (15 downto S1) → non-static signal name

Concurrent signal assignment (Syntax & Example)

Conditional Signal Assignment

- ENTC_S1
- <= "0001" when OPCD = "1101" else
- "0011" when OPCD = "1100" else
- "0111";

Concurrent signal assignment (Cont.)

Conditional Signal Assignment with delay

- ENTC_S2 <= "0001" after 10 ns when OPCD = "1101" ,
- ENTC_S1 after 20 ns when OPCD = "1100" ,
- not ENTC_S1 after 20 ns when OPCD = "1100" ,
- ENTC_S3 and ENTC_S1 after 50 ns when OPCD = "1111" ;

Concurrent signal assignment (Cont.)

Signal Assignment without condition

- ENTC_S4 <= (ENTC_S1 and ENTC_S2) xor ENTC_S3;

Selected Signal Assignment (case)

- Case ENTC_S1 is
  - When '0' => ENTC_S2 <= '1';
  - When others => ENTC_S2 <= '0';

Concurrent signal assignment (Cont.)

Assign an array of bit to an array of bits

- (S(0), A(2), C(5), ENTC_S1)
  - <= STD_LOGIC_VECTOR'("1010") after 5 ns,
  - STD_LOGIC_VECTOR'("0101") after 5 ns,
  - STD_LOGIC_VECTOR'("1100") after 5 ns;

Multiplier

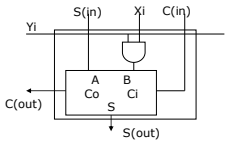
- Example: multiply 110 & 101;
  - 110
  - x 101
  - 110
  - 000
  - +110
  - 11110

Multiplier (cont.)

- How to implement it?
  - AND multiplicand with each bit of the multiplier
  - Produce the partial product
  - Align according to the position of the current bit of the multiplier
  - Repeat until do with the MSB of the multiplier
  - Add all the partial products

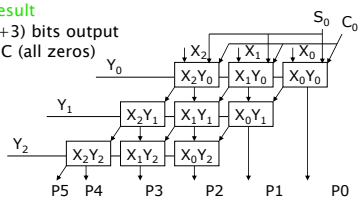
Multiplier: How to implement?

- Basic Mult. Block
  - AND two bits
  - ADD S(in) to Xi Yi with C(in)
  - Produce S(out) and C(out)



Multiplier: How to implement?

- Implement 3 x 3 multiplier
  - Cascade 3 basic multiplier block
  - Align the block cascaded
  - N x M multiplier can generate
  - up to N + M bit result
  - 3x3 → up to 6 (3+3) bits output
  - S0 & C0: initial S/C (all zeros)
  - X & Y: inputs;
  - Ps: Outputs



Writing in VHDL (Mult)

- Entity ← a design (or component)
  - Single bit multiplier
- List of ports
  - Inputs/outputs/inouts (single/array)
- Signals
  - Wires or connects ports and other components
- Note: ports are also signals

Writing in VHDL (Mult) (cont.)

- Architecture/Two Processes
  - Process: Describe an AND/a single-bit adder
  - Interconnect the two processes
- Verify your design
  - Simulation: using waveform captured
- Make 4-bit using the single-bit design
  - Similar, but structural design in VHDL

A VHDL single bit multiplier

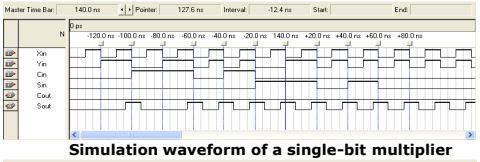
```
-------------------------------------------
--  A single bit multiplier for implementing a multiple bits multiplier  --
-------------------------------------------
library IEEE;
use IEEE.std_logic_1164.all;
use work.all;
entity single_bit_mult is
  port (Xin, Yin, Sin, Cin: in std_logic;
        Sout, Cout: out std_logic);
end single_bit_mult;
```

```
architecture a_single_bit_mult of single_bit_mult is
  :
  Declare signals/constants/components
  :
begin
  :
  Body of implementation
  :
end a_single_bit_mult;
```

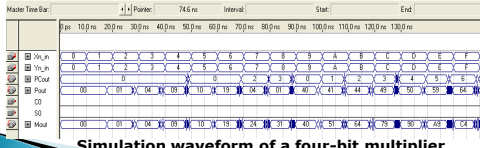

A VHDL four-bit multiplier (Partial Products)

```
-------------------------------------------------------------------------------
--      A four-bit multiplier (Partial Products) using single-bit multipliers          --
-------------------------------------------------------------------------------
library IEEE;
use IEEE.std_logic_1164.all;
use work.all;
entity partial_n_bit_mult is
  generic (i_length: positive := 4);        -- length of data
  port (Xn_in, Yn_in : in std_logic_vector (i_length-1 downto 0);
        S0, C0: in std_logic;
        PCout: out std_logic_vector (i_length-2 downto 0);
        Pout: out std_logic_vector (i_length*2-1 downto 0));
end partial_n_bit_mult;
architecture a_n_bit_mult of partial_n_bit_mult is
  component single_bit_mult                  -- component of the single-bit multiplier
    port (Xin, Yin, Sin, Cin: in std_logic;
          Sout, Cout: out std_logic);
  end component;
  signal s_S, s_C : std_logic_vector (i_length*i_length-1 downto 0);
begin
```


Sample Waveforms



Simulation waveform of a single-bit multiplier



Simulation waveform of a four-bit multiplier

VHDL Reference

- Ben Cohen "VHDL Coding Styles and Methodologies", Kluwer Academic Publishers